

Large Language Model Explainability for Planning Lunar Rover Trajectories under Uncertainty

Michelle Ho¹, Maximilian Adang¹, Zachary Booth², David E. Astle²,
Margaret Hansen⁴, Edward Balaban³, and Mykel J. Kochenderfer¹

Abstract—Planning planetary rover trajectories under probabilistic uncertainty is a multi-objective optimization problem that science teams currently iteratively solve and verify by hand. This process becomes impractical when timelines and constraints frequently shift throughout the mission planning phase. Sampling-based planners such as Monte Carlo Tree Search can speed up this process, but their outputs do not preserve traceability to the science objectives that justify each plan, which the science team needs to verify. We propose an interactive assistant based on a large language model (LLM) that can explain planner trajectories using domain vocabulary and re-run the planner under user-specified conditions to compare alternatives. Previous work has used LLMs inside planner decision-making loops, but we explore using them for post-planning explanation, where they are more likely to be trusted since they do not directly influence planner decisions. Our framework pairs a simulator inspired by the NASA Volatiles Investigating Polar Exploration Rover (VIPER) mission with a ReAct queryable interface that calls curated action tools and retrieval functions. We evaluate the assistant on a query bank consisting of plan comparisons, feasibility, and lookup queries. Experiments show that 90.9% of the assistant’s responses are faithful to the planner’s outputs, a nearly 65% improvement over zero-shot prompting of the same LLM without tool access, demonstrating the potential for trustworthy explainability of uncertainty-aware traverses with an LLM-based system.

I. INTRODUCTION

For space missions, planning the science phase typically requires vehicle trajectory optimization for multiple objectives and constraints. These trajectories also depend on many sources of probabilistic uncertainty, which become impractical to reiterate on quickly and frequently by hand. A simulator and an accompanying automated planning algorithm can speed up this iteration process, but their outputs may not be directly useful to the scientists who need to map these outputs to the effects on science data collection. This gap in explainability motivates an automated approach that uses large language models (LLMs) to explain trajectories in a comprehensible form to users who may not be familiar with the inner workings of the planning system.

Advances in robotics have been driven by LLMs applied to motion planning for tasks with high-dimensional observations, multi-modal sequences and behaviors, and complex failure modes [1]–[3]. For instance, recent work uses LLMs for planning Martian rover traverses [4], and efforts have begun to integrate LLMs with structured planners as state

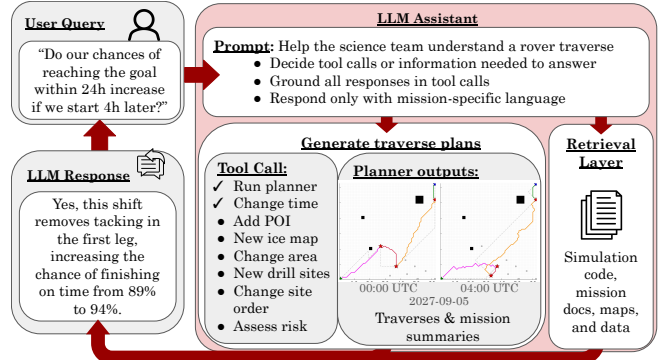


Fig. 1: End-to-end flow of our LLM-based mission planning assistant on an example query

trackers or plan verifiers within the planning stage [5], [6]. Improvements to LLM in-context reasoning, text generation, and information retrieval [7] suggest that they have the skills to provide comprehensive explainability for these tasks. However, current work does not use LLM reasoning capabilities to explain long-horizon plans made by rigorous techniques for planning under uncertainty.

One application that could benefit from this approach is planning planetary rover traverses. Such missions must account for a fixed time budget, navigation to sites of high scientific value, and uncertain speeds. We are inspired by NASA’s Volatiles Investigating Polar Exploration Rover (VIPER), which must explore a predefined area of high scientific value, called a science station, within a roughly 24-hour period. The rover is tasked with collecting measurements of volatiles with its three continuously operating spectrometers (neutron, near-infrared, and mass) through prospecting and obtaining subsurface regolith samples with its onboard drill [8]. The rover is solar-powered and thus needs to remain in sunlight as much as possible, unless operating within a permanently shadowed region (PSR) science station. The rover’s speed also depends on the sun’s position, as driving in certain directions is difficult or impossible due to the low-rising sun creating imaging issues for the navigation and hazard-avoidance cameras. The rover must also avoid communication shadows and slopes. Verifying by hand whether a handcrafted traverse satisfies all of these objectives and constraints is likely impractical, especially considering probabilistic uncertainties. An interpretable, automated planner is essential when timelines, constraints, and mission windows shift and require frequent replanning and verification.

NASA’s current tool for overseeing health-aware mission

¹Stanford University, Stanford, CA 94305 mtho@stanford.edu

²KBR Inc., ³Intelligent Systems Division, NASA Ames Research Center, Moffett Field, CA, 94035

⁴Carnegie Mellon University, Pittsburgh, PA 15213

planning is the System Health-Enabled Real-time Planning Advisor (SHERPA) [9] is a strategic mission planning AI system in use by NASA. SHERPA uses sampling-based methods like Monte Carlo Tree Search (MCTS) [10], [11] to compute and evaluate potential traverses for a variety of missions, such as VIPER’s long-range traverses. However, it currently does not have support for the detailed planning inside VIPER’s science stations. Additionally, MCTS plans trajectories by maximizing future rewards and rollouts, while the science team verifies trajectories against mission-specific objectives, such as risk of finishing over the time limit or maximizing prospecting area. The planner’s outputs do not directly trace back to these objectives, so it may require additional analysis to determine whether a given trajectory satisfies them.

To address this gap, we make the following contributions towards producing explainable trajectories:

- 1) An MCTS-based optimal trajectory planner coupled with a VIPER-inspired, physics-based simulator. Inputs to the planner include ice stability depth maps and sun-azimuth-dependent probability distributions for rover speeds.
- 2) An LLM-based, interactive assistant that dispatches the planner and a set of tools to automate replanning and explanation of trajectories with mission context, evaluated on a bank of science-team-inspired queries.

The proposed framework seen in Figure 1 is designed to allow the science team to efficiently replan under different conditions and constraints, while building trust with the trajectory planning algorithms, by keeping the team engaged in an interactive planning loop and grounding explanations in a familiar vocabulary. The LLM assistant does not override the planner, but rather runs it on behalf of the user with their specified conditions or explains resulting trajectories, without telling the user what they should do.

II. RELATED WORK

A. Sequential Decision Making under Uncertainty

Trajectory planning under uncertainty is often formulated as a Markov decision process (MDP) [11]. Since exact solutions to large MDPs are computationally intractable, online sampling-based methods are often used, such as Monte Carlo Tree Search (MCTS) [10], which builds a search tree of value estimates from simulated rollouts. We adopt a standard MDP formulation with an MCTS planner, as SHERPA does for strategic mission planning [9].

B. Rover Trajectory Planning

Rover planning has historically been a human-driven process. Gaines et al. [12] document the operational workflow for Mars rovers, where science teams hand-craft candidate traverses and validate them against constraints. They identify the cost of this loop as a primary limit on mission productivity. The process becomes even more complex in lunar polar missions, where a rover needs to avoid large, fast-moving illumination and communication shadows and where its speed, among other factors, depends on the difference between the

heading and the azimuth to the sun. The latter consideration is due to the sun, which does not rise more than a few degrees above the horizon, creating imaging issues for the rover’s navigation and hazard-avoidance cameras when traveling in certain directions. Another consideration in traverse planning is the achievement of the mission’s scientific objectives (e.g., detecting and characterizing deposits of water ice or other volatile materials and maximizing exploration within the area). To incorporate such constraints and considerations, the VIPER mission uses SHERPA [9], which synthesizes mission plans that optimize scientific productivity and maximize robustness to delays, malfunctions, and other adverse conditions using sampling-based solvers such as MCTS [10], [11]. However, the solver’s algorithmic framing may not be directly useful to the science team, who want to understand the solution within the context of science objectives. Our work targets this gap by using an LLM to explain the output from an MCTS-based planner that, coupled with a VIPER-inspired simulator, produces optimized rover trajectories.

C. LLMs for Planning

Previous work that integrates LLMs with planning has used them to define goals for task and motion planning [1], translate natural language task specifications into formal representations that planners consume [2], reason about motion failures and propose corrections [3], and serve as state trackers for long-horizon planners [5]. Additionally, recent work has placed LLMs inside MCTS itself, using them as a heuristic to guide the search [6]. LLMs in these algorithms drive planners within the decision loop to produce better trajectories, rather than being used to explain the resulting output trajectories after the planner is done.

One relevant area of LLM explainability can be found in work that leverages LLMs for reasoning about dynamic environments, specifically for automotive lane changing [13]. However, their application is limited to vehicle interaction modeling and does not consider interfacing LLMs with motion planning algorithms. Other works explore dynamic reasoning for trajectories [14] or combining MCTS with LLMs for self-training, evaluating intermediate reasoning steps for LLM outputs [15]. These approaches are promising for interfacing LLMs with MCTS but are directed at improving LLMs rather than using them to explain trajectories. Another application merges LLM reasoning with MCTS planning for dynamic behaviors in video games, where LLMs and MCTS work together to create emergent narratives [16]. While relevant to interfacing LLMs and MCTS, this work aims to improve agent behavior with LLMs, instead of using them for explainability.

D. Grounding and Tool Use for LLMs

Pretrained LLMs often perform worse on domain-specific tasks. Their broad training priors typically do not contain domain expert knowledge, so with zero-shot prompting, the model fills the gap with plausible-sounding but incorrect claims from its prior knowledge. Therefore, LLMs benefit from additional structure and knowledge for these tasks.

For instance, retrieval-augmented generation (RAG) [17] is a framework that grounds LLM responses in external data from a trusted source when queried. Additionally, tool-use frameworks treat the LLM as a controller that calls previously designed functions to inspect state or invoke external systems. ReAct (Reasoning and Acting framework) [18] allows the LLM to perform both reasoning and actionable tool calls. DSPy [19] puts this framework into a programmable module, so rather than relying on exact keyword matches for retrieval, the LLM decides which tools to call from declarative signatures. Unlike previous applications of these frameworks individually, our setting requires both in different roles. Retrieval without action tools could only describe prior runs, while tool access without retrieval could not translate between mission-language queries and planner outputs.

E. Explainable Planning

With the steady improvements in LLMs, there is existing work that also considers using them to build trustworthiness for AI-based planning. Previous work uses them to explain how task planners justify decisions to humans through plan explanations, contrastive “why not” queries, and model reconciliation [20]. These methods are tied to planners whose actions and states are discrete and labeled. They do not transfer to trajectory planners that operate over continuous, nondeterministic dynamics like the MCTS solver we use. Some prior work has also involved translating robot control policies into natural-language descriptions [21], having robots verbalize their plans before acting [22], and providing causal model grounded explanations for the behavior of model-free reinforcement learning agents [23]. However, though these efforts are meant to build trust between humans and autonomous systems, they explain the planner’s internal policy to a passive observer rather than the planner’s outputs to a domain expert without an algorithmic background.

III. METHODOLOGY

We formulate our VIPER-inspired trajectory planning problem within a science station as a sequential decision-making problem, and develop an MCTS-based planner that uses a simulator grounded in mission requirements and constraints. We then develop an LLM-based framework for providing accurate explanations of the planning results with a retrieval layer and tool catalog for a ReAct module.

A. Problem Formulation

We consider a rover that collects science data with contact measurements at a set number of drill sites and non-contact prospecting while traversing between the sites. The rover must complete all drills and reach a goal location before the station exit deadline, while also moving with an appropriate sun-azimuth-dependent speed made good (SMG) and avoiding obstacles such as craters.

Drill site selection and planning prospecting segments are handled in two stages. The set of sites to be visited consists of the points of interest (POIs) given by the science team and the set number of drill sites that the planner selects. This

selection considers a synthetic Gaussian process-based map that encodes information of interest. That information in this example is the scientific value of the ice volatiles. For a given location in the science station, each cell has an associated mean and variance [24]. This map structure is consistent with previous work that represents maps of natural phenomena for informative path planning as Gaussian processes [25]–[27].

We choose $N_c=10$ candidate drill sites on the map by a weighted sum of the mean and variance, balancing exploring sites with more uncertainty with sites that provide the highest scientific value. We use a minimum spacing constraint to spread them out over the map. We down-select to $K=3$ drill sites by enumerating all $\binom{N_c}{K} \cdot K!$ orderings and picking the lowest-expected-time feasible ordering under deterministic speed, giving the largest exploration budget for the robot to perform the most prospecting while traversing to drill sites.

Next, we formulate the problem of prospecting and navigating between sites under probabilistic speeds as a Markov decision process (MDP), represented by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ [11]. Rather than use a large state space and select actions to move between individual cells, we treat this as a semi-MDP (SMDP) [28], where we plan between nodes (start, goal, POIs, and drill sites) on a waypoint graph, $\mathcal{G}$. Thus, MCTS selects macro-actions, or which sites to visit in which order, and then we plan with A* to navigate in the cardinal and intercardinal directions through the cells between the nodes. The state s is the tuple $(n, \mathbf{d}, k, \ell, t)$ collecting the rover’s current waypoint node n , vector $\mathbf{d}$ with count k of visited sites, total path steps ℓ , and elapsed mission time t .

The transition dynamics function $T(s' | s, a)$ is nondeterministic, since the elapsed time depends on sampling the SMG distribution along the A* path between nodes. This models the various sources of execution uncertainty, such as localization and navigation errors, communication delays, wheel slip, or temporary malfunctions. The distribution is clipped such that the sampled SMG never exceeds the provided deterministic best-case value.

The SMG at each cell depends on the rover’s heading relative to the sun azimuth $\phi(t)$:

$$\Delta t \sim p_{\text{SMG}}(\cdot | \theta_{\text{rover}}(s), \phi(t)), \quad (1)$$

where Δt is the time required to traverse to the next cell, $\theta_{\text{rover}}(s)$ is the rover’s heading at state s , $\phi(t)$ is the sun’s azimuth at mission time t (obtained from a NASA ephemeris), and $p_{\text{SMG}}(\cdot | \theta_{\text{rover}}, \phi)$ is the empirical SMG distribution. Direct sunward travel requires tacking, or alternating between two oblique headings symmetric about the sunward direction due to the orientation of navigation and hazard-avoidance cameras. Turns add $\Delta t_{\text{turn}} = 107\text{s}$ to the elapsed time if they exceed a certain cone where the rover requires new images for the hazard map generation.

The reward function is defined as

$$R = R_{\text{event}} - \alpha \Delta t - \beta \tau^2, \quad (2)$$

where R_{event} is a reward for terminal events at s' (e.g., drilling a site or reaching the goal). The time-rate term Δt

penalizes traverse time on the required route. Minimizing the time required by the traverse directly maximizes the time available for prospecting, resulting in higher measurements coverage. The clipped fraction of the mission time budget elapsed at the next state s' is $\tau = \min\{t'/T_{\text{budget}}, 1.5\}$, squared and clipped so the penalty grows sharply while approaching the limit, but does not penalize further for going over. The weights for the time-related terms are $\alpha, \beta > 0$. Each drill operation takes 5.5 hours, leaving ~ 7.5 hours for traversing to the goal and prospecting in a 24-hour science station. Episodes terminate on mission success or time budget exhaustion, and we discount future rewards by $\gamma = 0.9$. Non-traversable terrain, like craters or rocks are treated as obstacles. We solve this MDP with an MCTS solver, with the hyperparameters to be included after publication.

B. Mission Planner

We instantiate the planner over an $n \times n$ grid with 1-m resolution. The SMG dependent on sun azimuth dictates that travel away from the sun is fastest, and sunward travel approaches zero unless the rover tacks. Pairwise A* paths between waypoints are precomputed once per problem instance. After MCTS returns the lowest-expected-time traverse, a post-processing stage greedily inserts prospecting detours into each waypoint-to-waypoint leg, using the slack between the conservative P90 traverse time and the mission budget to explore as much of the science station as is allowed. The planner outputs the chosen drill ordering, the enhanced trajectory, timing risk estimates, the extra prospected area, and the MCTS rollouts. This data is provided to the LLM assistant to provide comprehensive explanations to the user.

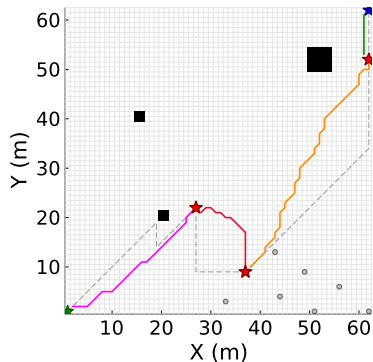


Fig. 2: Example MCTS SST for a science station within a 62 m x 62 m square. The planner picks 3 drill sites (red stars) from 10 candidates (gray circles), routes a path from the start (green star) through each drill, and to the goal (blue star). The most direct path (gray, dashed) is shown for reference. Each drill-segment (pink, red, orange) has prospecting detours injected with the remaining time budget. The segment to the goal is in green. Black squares mark untraversable terrain.

C. LLM Assistant

As seen in Figure 1, the assistant consists of three components that work with the planner: a chatbot user interface, a retrieval layer that grounds explanations in source material,

and a tool catalog that the LLM calls. The science team interacts with the assistant through a chat window where queries are typed and sent to the LLM agent. The framework parses the query and responds back in the chat window.

The retrieval layer gives the LLM the mission documents, planner source code (or the data), plots, and solutions that result from the planner as context when answering the user. Mission-document retrieval is implemented as a semantic search over chunked embeddings of the document. The source code and data retrieval are path lookups, so every cited fact is traceable to a specific file at a specific revision.

The chatbot is implemented with the ReAct framework [18] as a programmable DSPy module, allowing for more robust reasoning and acting than zero-shot prompting or retrieval alone can achieve. This structure requires a tool catalog that provides actions for the LLM to call (which align with possible operations that the science team may want to perform), listed here:

- **Run mission:** Plan a full mission from scratch and return the mission summary.
- **Set mission start:** Change the mission start date and time and replan, but reject start times whose 24-hour mission would put the sun below the horizon.
- **Add or replace POI:** Evaluate adding or replacing points of interest at specific coordinates. Returns feasibility and time impact.
- **Replan with new map:** Replan the mission using a different ice-map file.
- **Plan arbitrary region:** Plan a mission inside an arbitrarily shaped science region whose boundaries are defined by vertices given by the user.
- **Select custom drill sites:** Plan a mission using user-chosen drill sites from the candidate set. If infeasible, it suggests single-site swaps.
- **Force site order:** Replan with a user-specified site ordering rather than the planner’s chosen order.
- **Risk assessment:** Roll out the current plan N times under randomized, user-fixed, best case, or worst case speed uncertainty. Report the timing distribution (P50, P75, P90, P95, P99) and the success rate.

These actions allow the user to quickly replan with new conditions, make comparisons between potential plans, or improve understanding of the decisions made by the planner.

The LLM is instantiated with a prompt that takes the query as input and requires a natural-language response output. The LLM uses this prompt and the query to decide which tools and documents to call to generate the best response. The prompt to the LLM includes high-level guardrails to encourage responses that may resonate better with the science team, which will be made available upon publication. It constrains outputs to the operator vocabulary used by the science team (e.g., mission time budget, estimated ice quality, sun-relative SMG) and forbids algorithmic terminology (future rollouts, value estimates, reward functions). It also enforces that every factual claim in the response must trace either to a tool result or to the user query, not the model’s prior knowledge. The LLM assistant does not modify the underlying MDP or

override the planner. It can only run the planner on behalf of the user with their specified conditions or explain the results, without telling the user what they should do. A video demo will be made available upon publication.

D. Experiments

We design experiments to highlight the assistant’s contributions: (i) does the assistant correctly parse the user’s mission-language requests to identify the correct tools and information needed, and (ii) are the resulting explanations faithful to the planner’s outputs and to the source material? We report multiple metrics across multiple query categories and baselines/versions, with validation of any LLM-judged metric against human labels [29].

1) *Query Bank*: We construct a query bank of 115 science team questions, with the following categories:

- **Compare potential new plan**: Add or replace points of interest or drill sites, force a visit order, shift the mission start time, restrict planning to an arbitrary region, run multi-target science requests, and inspect individual segments,
- **Feasibility**: Evaluate confidence/risk of goal completion under probabilistic speed,
- **Mission-document and source code lookup**: Retrieval-only questions answered from the mission documents, the planner code and outputs, or descriptions of the assistant’s capabilities,
- **Ambiguous/Edge Cases**: Prompts where the LLM should request clarification (e.g., “is it safe?”, “make it faster”) or for impossible to answer queries (e.g., “place a drill site outside of the bounds”).

For each query, we record the LLM’s expected behavior (answer, clarify, or refuse), the expected tools the human expert would use, any data files the LLM should reference, and the expected human response in operator vocabulary.

2) *Testing Versions*: We compare four LLM assistant versions to be tested on the same query bank, sharing the same underlying generator LLM, decoding parameters, and vocabulary guardrails. The four versions highlight the contributions of the selected scaffolding, tools, and retrieval in the complete version. The versions are:

- **Zero-shot LLM**: The generator receives only the user’s query, with no system guardrails prompt, no tool catalog, and no retrieval layer,
- **Closed-book LLM**: The generator receives the user query and the guardrails prompt, but no tools and no retrieval, so it only relies on prior knowledge,
- **No-retrieval assistant**: ReAct with the action tools but without the retrieval layer,
- **Full assistant**: All retrieval and action tools with ReAct.

We do not have a retrieval-only version with action tools removed because that chatbot could only answer lookup-related questions and not any plan comparison queries.

3) *Success Metrics*: We organize our metrics into three categories. Trajectory metrics test task success on whether the LLM called the right tools with the right arguments. Extra

tool calls and extra optional arguments are allowed, but every oracle tool and every oracle argument value must be present. We also report passⁿ [30], the fraction of queries on which the assistant succeeds for all tested seeds. Task success is defined only for variants with tool access.

Semantic metrics test whether the natural-language response is accurate to the tool outputs and consistent with the human reference answer. For semantic accuracy, we report the fraction of factual claims in the response that are supported by the simulation data and tool outputs, ensuring the LLM is not favoring responding based on its prior knowledge. Because hand-labeling every response across all four versions and seeds is intractable at this scale, we evaluate responses on their semantic content with an LLM judge. We use a 1–5 G-Eval [31] score that considers consistency, relevance, and vocabulary against the human reference and pairwise win rate [32], measuring which version’s response the judge prefers when shown side by side. To validate the LLM judge, we hand-labeled a 50-query subset on semantic accuracy (yes/no/uncertain) and behavior, and we report Cohen’s κ [33] between judge and human on each axis. Cohen’s κ is a standard agreement metric that measures how much a judge and a human agree beyond what would happen by chance, penalizing a judge that defaults to the dominant label in the dataset. Following common practice in LLM-judge validation [34], we treat $\kappa < 0.6$ as a threshold below which we reject the judge’s categorizations. The judge is selected from a different model family than the generator to reduce self-preference bias [35].

Consistency metrics test whether semantically equivalent queries produce equivalent responses in the tools called and the semantic meaning. For queries whose expected behavior is to refuse or request clarification, we report whether the LLM executed the correct behavior. Each query is run independently through each variant for four seeds. We also note the monetary and latency costs of running each model. Reported numbers are means with 95% bootstrap confidence intervals over the query bank, computed per query category.

IV. RESULTS

A. Planner Behavior

Before evaluating the framework, we verify that the underlying planner responds to the inputs that the science team may want to vary: start time, selected POIs, the underlying ice map, the site visit order, the boundary area, and the probabilistic SMG. Figure 3 shows that each of these inputs changes the resulting plan in ways that match physical intuition. Delaying the start time by 12 hours changes both the route and the chosen drill set, since the planner might need to choose a different route to compensate for different SMG values. With our 200-rollout risk assessment, we observe that the mission durations cluster slightly above the deterministic estimate of 20.15 hours, with a small tail extending toward the 24-hour limit, a behavior consistent with the clipped SMG distribution. Across the rollouts, 99.5% finished within the 24-hour limit. Inserting 2 POIs correctly changes the trajectory to route through. Replanning with a different ice

map changes the drill candidates and chosen drill sites, therefore changing the trajectory. Enforcing a non-optimal drill order produced a 24.45-hour trajectory, exceeding the time limit, which was a reasonable result. Finally, we verified that the planner was able to plan within an arbitrarily defined science station area, given new start and end locations. Verifying that these results are reasonable for the planner, we then evaluate the LLM assistant’s ability to call the correct tools and information to explain the solutions.

B. End-to-End Evaluation

Table I reports the four-framework versions on the metrics defined in Section III-D.3. Comparing against an expert-labeled dataset, trajectory metrics score whether the assistant selects the correct tools, documents, and behavior (answer, clarify, refuse), and semantic metrics score whether the response is accurate to the output trajectories and external documents provided. We use GPT-5.4 and Claude Opus 4.8 as they represent current frontier models from two distinct families. Across both chatbot/judge combinations of GPT and Opus, the full assistant (with the guardrail prompt, retrieval layer, and action tools) achieved the highest score on every metric except G-Eval (vocabulary), or how well the response reuses the science team’s mission terminology. The zero-shot baseline, without tools or retrieval, can only ground its responses in the terminology from the query, whereas the other versions also bring in additional information surfaced by tool calls and retrieval. Vocabulary alone is not a comprehensive metric. The full framework outperforms on G-Eval (consistency) and (relevance), which measure whether the response is internally coherent and addresses the question asked, respectively. It also calls the correct tools with the correct arguments more often than the no-retrieval version, as shown with task success and passⁿ, and had a higher behavior accuracy, though just barely above zero-shot, since most of the queries called for an answer, and zero-shot rarely refuses to answer or asks for clarification. The full version’s responses are also preferred more often than the other versions’ responses. For pairwise win rate, the Opus judge prefers the full assistant to closed-book 70.0% of the time and to no-retrieval 65.4%. The GPT judge preferred full to closed-book 85.2% of the time. Moreover, as expected, the full version’s responses were more correct than the other versions, as it had access to all tools and documents.

We tested both combinations of chatbot/judge, with GPT-5.4 as chatbot and Opus 4.8 as judge, and vice versa. The full-version outperforms on all metrics except G-Eval (vocabulary) for both combinations, but notably, the Opus chatbot outperforms GPT on all metrics except for faithfulness. From a qualitative examination of the prompts, Opus’s responses were more verbose, so even though there were correct facts, it often added fabricated details as well. For instance, we posed the query “How much margin do we have under the 24-hour budget?” GPT correctly answered, “The current plan finishes in 20.15 hours, leaving 3.85 hours of margin under the 24-hour budget for prospecting,” whereas Opus added on a per-leg breakdown of slack and claimed that

“the drill-3 leg contributed the largest share of the buffer at 1.4 hours,” a number not given by any tool output. Though Opus outperforms in the other metrics, faithfulness is likely the most important metric. For example, for the trajectory metrics, the assistant may use different tools than expected, but still get the correct response, and the G-Eval metrics are more valuable if accompanied by accurate responses. Secondly, Opus rarely refuses to answer or ask for more clarification, so when trying the no-retrieval Opus chatbot version, it repeatedly attempted to call retrieval tools it did not have access to, exhausting tokens, time, and consistently errored before producing a hallucinated response.

TABLE II: Cohen’s κ between LLM judge and author labels.

judge	κ	agreement (%)
Opus 4.8	0.621	87.8
GPT-5.4	−0.095	50.0

The judge’s assessments are only trustworthy if they are aligned with a human expert’s judgments. To validate the LLM judges, we hand-labeled a subset of 50 responses for faithfulness and behavior (answer/clarify/refuse). Calculating Cohen’s κ [33] as seen in Table II, we see that Opus 4.8 is the better judge ($\kappa = 0.621$, with 87.8% agreement between LLM judge and human preference) compared to GPT-5.4 ($\kappa = -0.095$ with 50.0% agreement). Since $\kappa_{\text{opus}} \geq 0.6$ [34], we can rely on the judge’s scores as a proxy for human-validated checks on the responses. Therefore, with this validation and high faithfulness score with GPT chatbot, we consider the better pairing to be GPT as the chatbot and Opus as the judge. This is further supported by the comparison of time and token monetary costs incurred by each model as chatbots. As seen in Table III, it is evident that Opus requires much more time and costs nearly 20 times more than GPT. Future work will compare these expenditures against having the science team perform their validation.

Finally, to evaluate robustness to query phrasing, we ran semantically equivalent queries on the full assistant. There was consistent accuracy between single-fact lookups (e.g., “How long was the mission duration?”), specified a run with new conditions (e.g., “Insert a POI at (62, 1)”), or risk assessments (e.g., “How confident are we that we’ll make it to the goal on time?”). There were inconsistencies when the answer asked for just an explanation, rather than retrieving or running a tool. For instance, near-synonyms such as asking for how much buffer, slack, or remaining margin returned the exact value correctly, but the assistant provided supporting math only for slack and margin. Queries like “how does X work?” or “why does the path do Y ?” sometimes produced different explanations. For example, asking variations of “how does the planner weigh ice concentration vs. uncertainty?” either gave vague guesses (e.g., incorrectly claimed that the planner prioritizes value and only uses uncertainty as a tiebreaker) or the code’s correct formula. With these semantic inconsistencies with accurate values, we can address how the LLM chooses to phrase its response through DSPy prompt optimization [19]. The queries that failed faithfulness

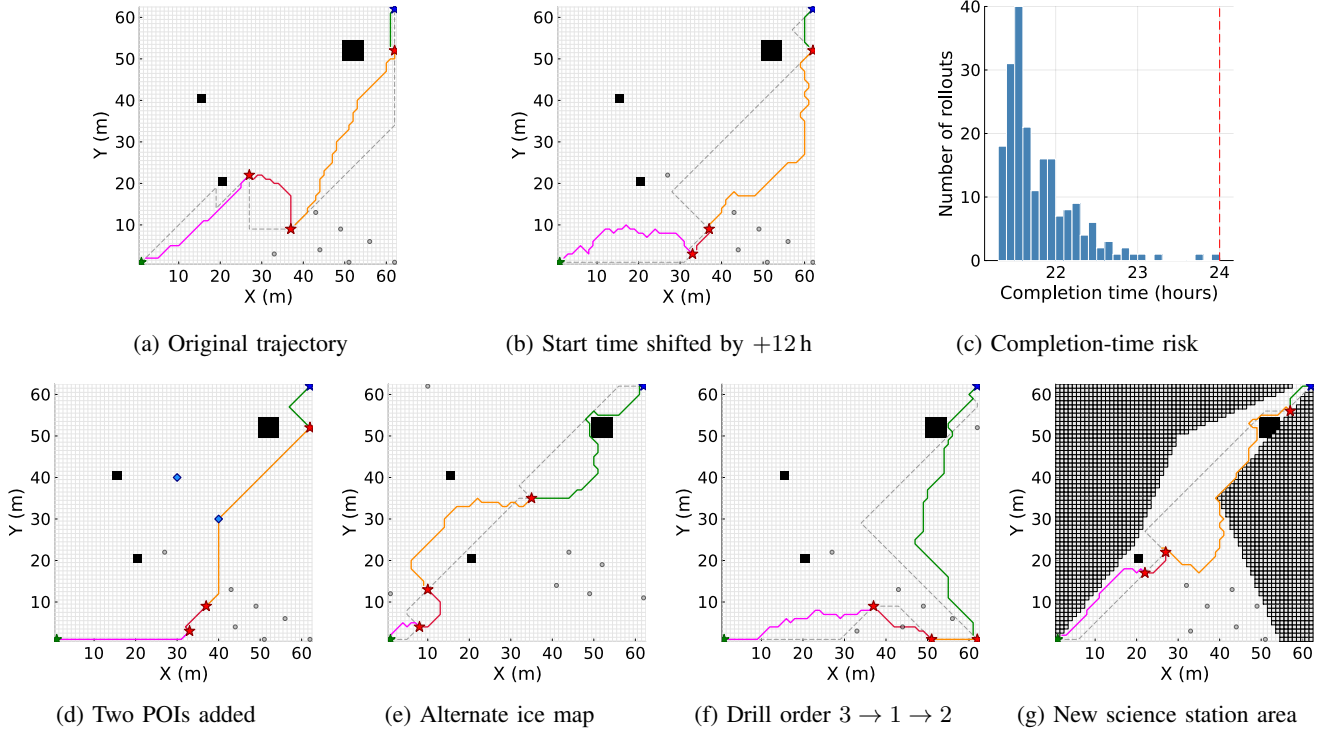


Fig. 3: Planner behavior demonstrated on different scenarios. All panels share the same MDP formulation, number of drill sites selected from 10 candidates, and MCTS hyperparameters. They use the same legend from Figure 2, with blue diamonds as POIs. (a) The original plan takes 20.15 hours without prospecting. (b) A +12h shift in start time results in a new plan due to different SMGs. (c) 200 Monte Carlo rollouts of (a) under SMG uncertainty, with 99.5% finishing in 24 hours. (d) We added 2 POIs, finishing in 21.7h. (e) We reran the planner with a new ice map. (f) Enforcing a non-optimal site ordering increased the mission time to 24.45 hours, exceeding the limit. (g) We replanned under new science station boundaries.

TABLE I: Results of chatbot/judge pairs for different framework versions across trajectory and semantic metrics on the query bank. Mean values are shown, and 95% bootstrap confidence intervals are non-overlapping for every comparison reported. Trajectory metrics (task success, pass⁴) are defined only for variants with a tool catalog.

Metric	Chatbot: GPT-5.4		Judge: Opus 4.8		Chatbot: Opus 4.8		Judge: GPT-5.4
	zero shot	closed book	no retrieval	full	zero shot	closed book	full
<i>Trajectory</i>							
task success	—	—	20.9	46.8	—	—	52.6
pass ⁿ	—	—	19.8	41.9	—	—	48.8
behavior acc.	78.0	33.7	50.0	79.1	80.4	26.5	84.8
<i>Semantic</i>							
faithfulness	26.7	56.1	72.6	90.9	23.4	10.1	65.3
G-Eval (consistency)	48.2	47.8	51.8	59.8	47.3	45.6	60.4
G-Eval (relevance)	75.2	65.1	71.6	85.1	80.9	59.6	88.2
G-Eval (vocabulary)	79.2	64.3	68.5	75.8	79.6	59.5	70.5

were on topics that were implemented in the planner, but had no grounding in the retrieved documents, such as why we require tacking. The full assistant’s 90.9% faithfulness is a nearly 65% improvement over the 26.7% zero-shot baseline. While hallucinations are a risk of any LLM system, we can improve through interactive sessions with the science team, addressing topics like tacking that are implemented but not discussed in the retrieved documentation.

V. CONCLUSION

We presented an LLM-based framework for explaining lunar rover trajectories planned under uncertainty, with an

MCTS planner accompanied by a VIPER-inspired simulator that includes probabilistic SMGs, a 24-hour mission budget, and exploration vs. exploitation tradeoffs. The full framework with action tools and retrieval outperforms the zero-shot baseline, with a 90.9% accuracy rate of responses. Our work has several opportunities for future improvement. We can extend the MDP to handle partial observability and incorporate active uncertainty reduction during trajectory planning. Additionally, we would like to test other solvers and use cases beyond MCTS and VIPER SST. The code repository for this work is available at <https://michelleho-24.github.io/rover-explainability/>.

TABLE III: Per-query cost and latency for both chatbots

version	lat. (s)	tokens	\$/query	total \$
GPT-5.4 chatbot				
zero_shot	5.27	15,903	0.0028	1.29
closed_book	1.32	884	0.0010	0.46
no_retrieval	9.02	7,738	0.0042	1.92
full	10.83	19,027	0.0086	3.97
chatbot subtotal	—	—	—	7.63
Opus 4.8 chatbot				
zero_shot	14.34	26,464	0.1628	74.90
closed_book	5.77	1,434	0.0139	6.38
no_retrieval	170.46	11,690	0.0835	0.50
full	29.77	31,277	0.1986	91.37
chatbot subtotal	—	—	—	173.15

ACKNOWLEDGMENTS

This work was supported by the NASA VIPER mission (Science Mission Directorate) and Amazon AI Fellowship.

REFERENCES

- [1] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, “Task and motion planning with large language models for object rearrangement,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2086–2092.
- [2] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, “AutoTAMP: Autoregressive task and motion planning with LLMs as translators and checkers,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6695–6702.
- [3] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu, “LLM3: Large language model-based task and motion planning with motion failure reasoning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [4] NASA Jet Propulsion Laboratory, “NASA’s Perseverance rover completes first AI-planned drive on Mars,” 2026.
- [5] S. Chen, A. Xiao, and D. Hsu, “LLM-State: Open world state representation for long-horizon task planning with large language model,” in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Vision-Language Models for Navigation and Manipulation*, 2024.
- [6] Z. Zhao, W. S. Lee, and D. Hsu, “Large language models as commonsense knowledge for large-scale task planning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 31 967–31 987.
- [7] W. X. Zhao, K. Zhou, J. Li, T. Tang, Z. Dong, Y. Hou, B. Zhang, Y. Min, J. Zhang, P. Liu, X. Wang, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, Y. Hu, J.-Y. Nie, and J.-R. Wen, “A Survey of Large Language Models,” *Frontiers of Computer Science*, vol. 20, no. 12, p. 2012627, 2026.
- [8] A. Colaprete, “Volatiles Investigating Polar Exploration Rover (VIPER),” NASA Ames Research Center, Tech. Rep. 20210015009, 2021.
- [9] E. Balaban, Z. Booth, E. Astle, M. Ho, H. Warner, G. Kim, D. Eddy, S. Banerjee, J. Ott, M. Shirley, K. Bradner, and A. Colaprete, “SHERPA — An artificial intelligence system for mission planning and decision support,” in *International Conference on Space Operations (SpaceOps)*, 2025.
- [10] R. Coulom, “Efficient selectivity and backup operators in Monte-Carlo tree search,” in *Computers and Games*. Springer, 2006, pp. 72–83.
- [11] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for Decision Making*. MIT Press, 2022.
- [12] D. Gaines, R. Anderson, G. Doran, W. Huffman, H. Justice, R. Mackey, G. Rabideau, A. Vasavada, V. Verma, T. Estlin, L. Fesq, M. Ingham, M. Maimone, and I. Nesnas, “Productivity challenges for Mars rover operations,” in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016, pp. 115–125.
- [13] M. Peng, X. Guo, X. Chen, K. Chen, M. Zhu, L. Chen, and F.-Y. Wang, “LC-LLM: Explainable lane-change intention and trajectory predictions with large language models,” *Communications in Transportation Research*, vol. 5, p. 100170, 2025.
- [14] Y. Zhang, “Unveiling LLM mechanisms through neural ODEs and control theory,” *arXiv preprint arXiv:2406.16985*, 2024.
- [15] D. Zhang, S. Zhoubian, Z. Hu, Y. Yue, Y. Dong, and J. Tang, “ReST-MCTS* : LLM self-training via process reward guided tree search,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 64 735–64 772.
- [16] Q. Guignard, “Exploring LLMs and MCTS for emergent narrative,” Master’s thesis, ETH Zurich, 2024.
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [19] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, “DSPy: Compiling declarative language model calls into state-of-the-art pipelines,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [20] T. Chakraborti, S. Sreedharan, and S. Kambhampati, “The emerging landscape of explainable automated planning & decision making,” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2020, pp. 4803–4811.
- [21] B. Hayes and J. A. Shah, “Improving robot controller transparency through autonomous policy explanation,” in *ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 2017, pp. 303–312.
- [22] S. Rosenthal, S. P. Selvaraj, and M. Veloso, “Verbalization: Narration of autonomous robot experience,” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2016, pp. 862–868.
- [23] P. Madumal, T. Miller, L. Sonenberg, and F. Vetere, “Explainable reinforcement learning through a causal lens,” in *AAAI Conference on Artificial Intelligence (AAAI)*, vol. 34, no. 03, 2020, pp. 2493–2500.
- [24] M. Hansen and D. Wettergreen, “Learning physically informed maps for off-world autonomous scientific discovery,” in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Space Robotics and Autonomy*, 2026.
- [25] Y. Brouwer, A. Vale, and R. Ventura, “Informative path planner with exploration–exploitation trade-off for radiological surveys in non-convex scenarios,” *Robotics and Autonomous Systems*, vol. 136, p. 103691, 2021.
- [26] G. Hitz, E. Galceran, M.-E. Garneau, F. Pomerleau, and R. Siegwart, “Adaptive continuous-space informative path planning for online environmental monitoring,” *Journal of Field Robotics*, no. 8, pp. 1427–1449, 2017.
- [27] A. Candela, D. Thompson, E. N. Dobrea, and D. Wettergreen, “Planetary robotic exploration driven by science hypotheses for geologic mapping,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 3811–3818.
- [28] M. Baykal-Gürsoy and K. Gürsoy, “Semi-markov decision processes: Nonstandard criteria,” *Probability in the Engineering and Information Sciences*, vol. 21, no. 4, p. 635–657, 2007.
- [29] R. Bommasani, P. Liang, and T. Lee, “Holistic evaluation of language models,” *Annals of the New York Academy of Sciences*, vol. 1525, no. 1, pp. 140–146, 2023.
- [30] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” *arXiv preprint arXiv:2406.12624*, 2024.
- [31] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 2511–2522.
- [32] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, E. P. Xing, J. E. Gonzalez, I. Stoica, and H. Zhang, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [33] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [34] S. Tan, S. Zhuang, K. Montgomery, W. Y. Tang, A. Cuadron, C. Wang, R. A. Popa, and I. Stoica, “JudgeBench: A benchmark for evaluating LLM-based judges,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [35] A. Panickssery, S. R. Bowman, and S. Feng, “Llm evaluators recognize and favor their own generations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 68 772–68 802.